

JavaScript Error Statements

[Back to JS page](#)

Table of Contents

- [JavaScript Error Statements](#)
 - [The try and catch Blocks](#)
 - [Example 1](#)
 - [The Error Object](#)
 - [Example 2](#)
 - [The finally Block](#)
 - [Example 3](#)
 - [The throw Statement](#)
 - [Example 4](#)
 - [Input Validation](#)
 - [Example 5](#)
 - [Error Object Properties](#)
 - [Example 6](#)
 - [Document](#)
 - [Reference](#)

The try and catch Blocks

The `try` statement defines a code block to run (to try). The `catch` statement defines a code block to handle any error. If an error occurs in the `try` block, the `catch` block is executed.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Error Statements</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Try and Catch</h4>
<p id="demo"></p>

<script>
try {
  adddler("Welcome guest!");
}
catch(err) {
  document.getElementById("demo").innerHTML = err.message;
}
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The Error Object

When an error occurs, JavaScript creates an **Error object** with two properties: `name` and `message`. The `name` property returns the error type (e.g., `ReferenceError`, `TypeError`). The `message` property returns a description of the error.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Error Statements</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Error Object</h4>
<p id="demo"></p>

<script>
let text = "";

try {
  adddler("Welcome guest!");
}
catch(err) {
  text += "Error Name: " + err.name + "<br>";
  text += "Error Message: " + err.message;
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

The finally Block

The `finally` statement lets you execute code after `try` and `catch`, regardless of the result. The `finally` block will always run, whether an error occurred or not.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Error Statements</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The finally Block</h4>
<p id="demo"></p>

<script>
let text = "";

try {
  adddler("Welcome guest!");
}
catch(err) {
  text += "Error caught: " + err.message + "<br><br>";
}
finally {
  text += "The 'finally' block always runs.";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

The throw Statement

The `throw` statement allows you to create a custom error. You can throw a string, number, boolean, or an object. When you use `throw` together with `try` and `catch`, you can control program flow and generate custom error messages.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Error Statements</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The throw Statement</h4>
<p id="demo"></p>

<script>
let text = "";

try {
    throw "Too big";    // throw a text
}
catch(err) {
    text += "Error: " + err;
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Input Validation

The `throw` statement is commonly used for input validation. If the input is invalid, you can throw an error message and catch it to display to the user.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Error Statements</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Input Validation</h4>
<p>Please input a number between 5 and 10:</p>

<input id="demo" type="text">
<button type="button" onclick="myFunction()">Test Input</button>
<p id="message"></p>

<script>
function myFunction() {
  const message = document.getElementById("message");
  message.innerHTML = "";
  let x = document.getElementById("demo").value;

  try {
    if(x.trim() == "") throw "empty";
    if(isNaN(x)) throw "not a number";
    x = Number(x);
    if(x < 5) throw "too low";
    if(x > 10) throw "too high";
  }
  catch(err) {
    message.innerHTML = "Input is " + err;
  }
}
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Error Object Properties

The Error object provides several properties to help debug and identify problems:

- **name:** The type of error (e.g., "ReferenceError", "TypeError")

- **message:** A human-readable description of the error

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Error Statements</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Error Object Properties</h4>
<p id="demo"></p>

<script>
let text = "";

// ReferenceError
try {
    x = y + 1;
}
catch(err) {
    text += "Error Name: " + err.name + "<br>";
    text += "Error Message: " + err.message + "<br><br>";
}

// TypeError
try {
    let num = 1;
    num.toUpperCase();
}
catch(err) {
    text += "Error Name: " + err.name + "<br>";
    text += "Error Message: " + err.message;
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Error Statements](#)